

MedAssist — Assistente Médico Inteligente com IA

Fine-tuning de LLM + RAG MedQuAD + LangChain + LangGraph

FIAP Pós-Tech — Inteligência Artificial para Devs

Tech Challenge — Fase 3
Maio / 2026

Requisito	Implementação	Arquivo
Fine-tuning de LLM com dados médicos	Phi-3-mini-4k + QLoRA 4-bit (Unsloth)	MedAssist_FineTuning_Colab.ipynb
Preprocessing e curadoria de dados	MedQuAD: 5.274 pares PT-BR (NIH)	medquad_dataset/ + fase1_dados/
Pipeline LangChain com LLM customizada	RAG: FAISS + MiniLM-L12 + MedQuAD	fase3_langchain/chains.py
Fluxos de decisão automatizados	LangGraph — 6 nós com roteamento	fase3_langchain/graph.py
Segurança — guardrails clínicos	Safety Guard + 5 cenários aprovados	fase4_seguranca/
Logging e auditoria	JSON por interação em logs/audit.log	fase4_seguranca/logging_audit.py
Explainability das respostas	Fontes MedQuAD + score por resposta	fase4_seguranca/explainability.py
Interface web profissional	FastAPI + chat + fontes + modal backend	web_app.py + web_ui.html
Código modularizado + README	5 fases Python + instruções completas	README.md
Avaliação do modelo	Métricas ROUGE, BLEU, F1 + relatório	fase5_avaliacao/

1. Introdução e Contexto

Este projeto implementa um assistente virtual médico desenvolvido para o Tech Challenge Fase 3 da POSTECH/FIAP. O desafio propõe criar um assistente treinado com dados próprios, capaz de auxiliar em condutas clínicas e responder dúvidas com base em protocolos médicos, organizando fluxos de decisão automatizados e seguros coordenados com LangChain.

O MedAssist integra quatro pilares tecnológicos complementares: (1) fine-tuning real de LLM com dados médicos em português usando QLoRA, (2) sistema RAG sobre 5.274 amostras do MedQuAD indexadas em FAISS, (3) orquestração de fluxos automatizados com LangChain e LangGraph, e (4) guardrails de segurança clínica com logging de auditoria e explainability das respostas.

1.1 Requisitos Atendidos

Requisito	Implementação	Arquivo
Fine-tuning de LLM com dados médicos	Phi-3-mini-4k-instruct + QLoRA 4-bit (Unsloth)	fase2_finetuning/MedAssist_FineTuning_Colab.ipynb
Dataset: preprocessing, anonimização, curadoria	MedQuAD PT-BR — 5.274 pares Q&A (NIH)	medquad_dataset/ + fase1_dados/
Pipeline LangChain com LLM customizada	RAG: FAISS + MiniLM-L12 + MedQuAD indexado	fase3_langchain/chains.py
Consultas em base de dados estruturada	Vector store FAISS — top-3 por consulta	outputs/vectorstore/
Contextualização das respostas	Contexto RAG injetado no prompt antes do LLM	fase3_langchain/chains.py + graph.py
Fluxos de decisão automatizados (LangGraph)	6 nós: entrada → triagem → RAG → geração → validação	fase3_langchain/graph.py
Limites: não prescrever sem validação humana	Safety Guard com flag + aviso obrigatório	fase4_seguranca/safety_guard.py
Logging para rastreamento e auditoria	JSON por interação em logs/audit.log	fase4_seguranca/logging_audit.py
Explainability — fonte da informação	Fontes MedQuAD + score de confiança	fase4_seguranca/explainability.py
Projeto modularizado + README completo	5 fases + fluxo sequencial + troubleshooting	README.md

2. Fine-tuning de LLM — Phi-3-mini + QLoRA

O fine-tuning foi realizado no Google Colab com GPU T4 gratuita, usando a técnica QLoRA (Quantized Low-Rank Adaptation) via biblioteca Unsloth. A escolha do Phi-3-mini-4k-instruct (3.8B parâmetros) foi estratégica: modelo compacto com excelente suporte a chat template, instruções e português, que cabe no T4 com quantização 4-bit.

2.1 Representação e Dataset de Treinamento

O dataset de fine-tuning consiste em pares de perguntas e respostas médicas em português brasileiro, extraídos e processados do MedQuAD (NIH). Cada amostra segue o formato de chat com três turnos: system (prompt médico), user (pergunta) e assistant (resposta). O modelo aprende a responder perguntas clínicas no contexto do prompt de sistema, que o instrui como assistente médico.

Divisão	Quantidade	Proporção
Dataset total (PT-BR)	5.274 pares Q&A	100%
Treino (train.jsonl)	4.747 amostras	90%
Validação (val.jsonl)	527 amostras	10%
Formato por amostra	JSONL — messages: system / user / assistant	—

2.2 Técnica QLoRA e Parâmetros

QLoRA combina quantização 4-bit (NF4) do modelo base com adaptadores Low-Rank (LoRA) treináveis. O modelo base é congelado e apenas os adaptadores são treinados, reduzindo o requisito de VRAM de ~16 GB para ~6 GB — viabilizando o treinamento no T4. A biblioteca Unsloth otimiza ainda mais o throughput em 30-50% em relação ao treinamento padrão com Hugging Face.

Parâmetro / Operador	Técnica / Valor	Detalhe
Modelo base	Phi-3-mini-4k-instruct (3.8B)	Microsoft — suporte nativo a português e chat
Técnica de fine-tuning	QLoRA 4-bit (NF4) via Unsloth	Reduz VRAM de ~16 GB para ~6 GB
LoRA rank / alpha	r=16 / α =32	Balanco entre capacidade e eficiência
Target modules	q/k/v/o_proj, gate/up/down_proj	Camadas de atenção + FFN — 7 módulos
Épocas	1 (~45 min no T4)	Suficiente sem overfitting para corpus médico
Batch size efetivo	16 (grad. accum. 4 × batch 4)	Simula batch maior com pouca VRAM
Learning rate	2e-4	Padrão QLoRA recomendado
Otimizador	AdamW 8-bit	Reduz consumo de memória do otimizador
Scheduler	Cosine	Decaimento suave do learning rate

Parâmetro / Operador	Técnica / Valor	Detalhe
Quantização base	NF4 4-bit com double quant	bitsandbytes — compute dtype float16

2.3 Critério de Parada e Validação de Saída

O treinamento monitora a loss de validação a cada epoch. Com 1 epoch sobre 4.747 amostras, a loss de treinamento cai de ~2.1 para ~0.8, indicando convergência sem overfitting. Ao final, os adaptadores LoRA são salvos em outputs/model/ (~114 MB). Por exceder o limite do Git, o arquivo adapter_model.safetensors está disponível via Google Drive (link no README).

A validação dos adaptadores é feita localmente pelo script validate_adapters.py, que executa 28 verificações automatizadas:

Grupo de Verificação	Verificações	Resultado
Arquivos obrigatórios	adapter_config.json, adapter_model.safetensors, tokenizer.*, chat_template.jinja	✓ OK
Tamanho do safetensors	Mínimo 50 MB esperado	✓ OK — 114.1 MB
Configuração LoRA	peft_type=LORA, task_type=CAUSAL_LM, r=16, α=32	✓ OK
Target modules	7 módulos: q/k/v/o_proj, gate/up/down_proj	✓ OK
Estrutura dos tensores	448 tensores — 224 pares lora_A/lora_B	✓ OK
Prefixo dos tensores	base_model.model.*	✓ OK
Tokenizer funcional	vocab_size=32.064, tokenizer.model presente	✓ OK
Chat template	chat_template.jinja presente e válido	✓ OK
inference_mode	True — adaptadores prontos para inferência	✓ OK

Resultado: APROVADO 28/28 verificações — execute: python fase2_finetuning/validate_adapters.py

2.4 Avaliação Qualitativa no Colab — 5/5 aprovadas

Pergunta de Teste	Resultado Obtido	Avaliação
Sintomas do diabetes tipo 2	8 sintomas listados + orientação de consulta	✓ Aprovado
Causas da hipertensão arterial	8 fatores causais com explicação clínica	✓ Aprovado
Como funciona a quimioterapia	Mecanismo, vias de administração, efeitos colaterais	✓ Aprovado
Sintomas de AVC	6 sinais de alerta + urgência imediata (SAMU)	✓ Aprovado
O que é leucemia linfoblástica aguda	Definição clínica completa com subtipos	✓ Aprovado

2.5 Decisão Arquitetural sobre Inferência

O Phi-3-mini requer ~8 GB de VRAM para inferência. Para demonstração e desenvolvimento sem GPU local, optou-se pela seguinte arquitetura:

- Fine-tuning comprovado no Colab: adaptadores LoRA em outputs/model/ validam o treinamento real
- MODO 1 — OpenAI API (gpt-4o-mini): inferência rápida sem GPU, ideal para demonstração
- MODO 2 — Ollama + Gemma 3 (gemma3:4b): inferência local sem API key, funciona offline
- MODO 3 — Ollama + Phi-3-mini fine-tunado (phi3-medassist): modelo do Colab convertido para GGUF (phi-3-mini-4k-instruct.Q4_K_M.gguf, ~2.2 GB), registrado no Ollama — sem GPU, sem API key, máxima especialização médica

Essa arquitetura desacopla o fine-tuning da inferência, permitindo demonstrar o pipeline completo em qualquer máquina — incluindo o próprio modelo treinado via MODO 3.

3. Pipeline de Dados — MedQuAD PT-BR

O MedQuAD (Medical Question Answering Dataset) é um corpus público do NIH (National Institutes of Health) com pares de perguntas e respostas extraídos de 12 categorias de fontes médicas oficiais. É um dos datasets sugeridos diretamente pelo enunciado do Tech Challenge Fase 3.

3.1 Fontes e Estrutura do Dataset Original

Pasta	Fonte	Domínio Médico
01_MedlinePlus	MedlinePlus (NIH)	Saúde geral e condições comuns
02_GARD_QA	GARD (NIH)	Doenças raras
03_GHR_QA	Genetics Home Reference	Condições genéticas hereditárias
04/10_MPlus_ADAM	MedlinePlus ADAM	Enciclopédia médica geral
05_NIDDK	NIDDK (NIH)	Diabetes, digestão, rim e urologia
06_NINDS	NINDS (NIH)	Doenças neurológicas
07_SeniorHealth	SeniorHealth	Saúde do idoso
08_NHLBI	NHLBI (NIH)	Coração, pulmão e sangue
09_CDC	CDC	Controle e prevenção de doenças
12_Cancer_QA	NCI (NIH)	Oncologia

3.2 Pipeline de Processamento — Etapas

Etapas	Script	Entrada	Saída
1. Clone do repositório	git clone (GitHub)	—	~16.000 arquivos XML brutos
2. Limpeza e filtragem	clean_medquad.py	XMLs brutos	8.653 pares EN (medquad_clean.jsonl)
3. Tradução PT-BR	translate_ptbr.py	8.653 pares EN	5.274 pares PT-BR (medquad_ptbr.jsonl)
4. Split treino/val	fase1_dados/preprocess.py	5.274 pares PT-BR	train.jsonl (4.747) + val.jsonl (527)
5. Exploração	fase1_dados/explore_dataset.py	train.jsonl	outputs/explore_report.txt
6. Validação	fase1_dados/validate_data.py	train + val	outputs/data_quality_report.txt

3.3 Filtros de Qualidade Aplicados (clean_medquad.py)

Filtro	Implementação	Motivo
Tamanho mínimo — pergunta	len(q) >= 10 caracteres	Remove perguntas degeneradas ou truncadas
Tamanho mínimo — resposta	len(a) >= 20 caracteres	Remove respostas vazias ou insuficientes

Filtro	Implementação	Motivo
Tamanho máximo — resposta	<code>len(a) <= 1.000 caracteres</code>	Remove respostas excessivamente longas
Conteúdo genérico inválido	<code>"contact your doctor" not in a</code>	Remove respostas sem valor informativo
Deduplicação	Hash por conteúdo da pergunta	Remove pares duplicados exatos
Limpeza de HTML	<code>re.sub(r'<[^>]+>', '', text)</code>	Remove tags HTML residuais dos XMLs
Normalização de espaços	<code>re.sub(r'\s+', ' ', text)</code>	Normaliza espaços em branco múltiplos

3.4 Tradução PT-BR (translate_ptbr.py)

A tradução foi realizada com a biblioteca `deep_translator` usando o motor Google Translate. Os campos traduzidos são: a pergunta do usuário (`user`) e a resposta do assistente (`assistant`). O prompt de sistema foi reescrito diretamente em PT-BR. Em caso de falha de tradução — `timeout` ou texto muito longo — o script mantém o texto original como fallback, garantindo que o dataset nunca fique incompleto.

Métrica	Valor
Total de pares limpos em inglês (entrada)	8.653
Total de pares traduzidos para PT-BR (saída)	5.274
Redução (falhas de tradução + filtro pós-tradução)	~39% descartados
Tempo estimado para tradução completa	2–4 horas (rate limiting Google Translate)
Ferramenta	<code>deep_translator</code> — <code>GoogleTranslator(source='en', target='pt')</code>

3.5 Estatísticas do Dataset Final

Métrica	Valor
Total de amostras PT-BR	5.274 pares Q&A
Amostras de treino (90%)	4.747
Amostras de validação (10%)	527
Comprimento médio — pergunta	~67 caracteres
Comprimento médio — resposta	~312 caracteres
Amostras sem resposta	0
Duplicatas no dataset final	0
Formato	JSONL — <code>messages: system / user / assistant</code>
Licença	Público — NIH/MedQuAD (uso acadêmico)

4. Assistente Médico — LangChain + LangGraph + RAG

4.1 Pipeline RAG (chains.py)

O pipeline RAG indexa o corpus MedQuAD PT-BR em um vector store FAISS usando embeddings sentence-transformers (MiniLM-L12). Para cada consulta, os 3 documentos mais relevantes por similaridade cosine são recuperados e injetados no contexto do prompt antes de chamar o LLM. Esse processo garante que as respostas sejam fundamentadas em informações médicas reais do corpus.

Componente RAG	Tecnologia	Configuração
Modelo de embeddings	sentence-transformers	paraphrase-multilingual-MiniLM-L12-v2
Vector store	FAISS (faiss-cpu)	outputs/vectorstore/index.faiss
Documentos recuperados (K)	Top-K por consulta	k=3 — contexto de 3 Q&A relevantes
Corpus indexado	MedQuAD PT-BR	5.274 pares Q&A médicos em português
Métrica de similaridade	Similaridade cosine	Via embeddings normalizados
Persistência	Disco local	index.faiss + index.pkl em outputs/vectorstore/

4.2 Fluxo LangGraph — Grafo de Estados (graph.py)

O LangGraph orquestra o fluxo completo de cada requisição médica usando um StateGraph com estado compartilhado (MedState). O estado carrega a pergunta, categoria, documentos recuperados, resposta bruta, resposta final, fontes e flags de segurança ao longo de todos os nós.

Nó	Função	Saída no Estado
node_entrada	Recebe e normaliza a pergunta do usuário	question: str normalizado
node_triagem	Detecta emergências, saudações e consultas normais via keywords	category: 'emergencia' 'fora_escopo' 'consulta'
node_emergencia	Bloqueia LLM — gera resposta com SAMU 192 / Bombeiros 193	raw_answer: str de emergência
node_busca_rag	Recupera top-3 documentos relevantes do FAISS	retrieved_docs: list[str] + sources: list[str]
node_geracao	Chama llm_backend.generate() com contexto RAG injetado	raw_answer: str gerado pelo LLM
node_validacao	Adiciona fontes, flags de prescrição e disclaimer médico	final_answer: str completo

Roteamento Condicional

O roteamento pós-triagem é implementado como edge condicional do LangGraph:

- Categoria 'emergencia' → node_emergencia (bypassa RAG e LLM)
- Categoria 'fora_escopo' ou 'consulta' → node_busca_rag → node_geracao

Esse design garante que situações de emergência nunca dependam do LLM — a resposta é determinística e imediata.

4.3 Backends de Inferência — llm_backend.py

O llm_backend.py unifica três modos de inferência com a mesma interface pública (função generate). A troca pode ser feita editando o .env ou pelo modal da interface web (3 opções) em tempo real, sem reiniciar o servidor FastAPI.

Modo	Backend	Configuração .env	Característica
MODO 1	OpenAI API (gpt-4o-mini)	LLM_BACKEND=openai OPENAI_API_KEY=sk-...	Rápido (~2s), sem GPU, custo por token
MODO 2	Ollama + Gemma 3 (gemma3:4b)	OPENAI_BASE_URL=http://localhost:11434/v1 OPENAI_MODEL=gemma3:4b	Gratuito, offline, sem GPU obrigatória
MODO 3	Ollama + Phi-3-mini fine-tunado (phi3-medassist)	OPENAI_BASE_URL=http://localhost:11434/v1 OPENAI_MODEL=phi3-medassist	Modelo treinado no Colab via GGUF — sem GPU, sem API key, máxima especialização médica

MODO 3 — pré-requisito: baixar phi-3-mini-4k-instruct.Q4_K_M.gguf (~2.2 GB) do Google Drive (link no README), colocar em outputs/gguf/ e executar: ollama create phi3-medassist -f Modelfile.

5. Segurança e Validação

O MedAssist implementa três camadas de segurança clínica, atendendo diretamente ao requisito 3 do Tech Challenge: guardrails que limitam a atuação do sistema, logging detalhado para rastreamento e auditoria, e explainability das respostas geradas.

5.1 Safety Guard — Guardrails Clínicos (safety_guard.py)

Guardrail	Trigger (palavras-chave)	Ação do Sistema
Emergência médica	infarto, avc, parada cardíaca, não respira, desmaio, convulsão, engasgo...	Bloqueia LLM → SAMU 192 e Bombeiros 193
Saúde mental crítica	suicídio, me matar, automutilação, vou me machucar, quero morrer...	Redireciona → CVV 188 (não passa pelo LLM)
Prescrição direta	que remédio tomar, me prescreva, me receite, qual antibiótico...	Flag de prescrição + aviso: revisão humana obrigatória
Fora do escopo	Saudações, tópicos não médicos, perguntas fora do domínio	Resposta padrão informando o escopo do sistema
Disclaimer médico	Toda e qualquer resposta gerada pelo LLM	Aviso automático: não substitui profissional de saúde

Testes automatizados: 5/5 cenários aprovados — execute: `python fase4_seguranca/safety_guard.py`

5.2 Logging e Auditoria (logging_audit.py)

Cada interação com o assistente é registrada em logs/audit.log em formato JSON Lines. O log inclui todos os campos necessários para rastreabilidade e auditoria regulatória em sistemas de saúde:

Campo do Log	Descrição
timestamp	Data e hora ISO 8601 da interação
session_id	Identificador único da sessão do usuário
question	Pergunta original do usuário
category	Categoria detectada: emergencia fora_escopo consulta
backend	Backend utilizado: openai ollama
model	Modelo específico: gpt-4o-mini gemma3:4b phi3-medassist
sources	Lista de fontes MedQuAD consultadas (top-3)
flags	Flags de segurança disparadas (prescrição, emergência)
response_time_ms	Tempo total de resposta em milissegundos
final_answer (hash)	Hash da resposta — para detecção de respostas duplicadas

5.3 Explainability (explainability.py)

Cada resposta inclui as fontes do MedQuAD consultadas durante a recuperação RAG, exibidas na interface web no painel de detalhes lateral. Um score de confiança é calculado com base na similaridade semântica média entre a pergunta e os documentos recuperados, permitindo que o usuário avalie a relevância da resposta.

Componente de Explainability	Implementação
Fontes consultadas	Top-3 perguntas MedQuAD mais próximas — exibidas por consulta
Score de confiança	Similaridade cosine média entre pergunta e documentos recuperados
Flags de segurança	Lista de avisos disparados (prescrição, disclaimer)
Visibilidade na UI	Painel lateral direito na interface web — fonte + score visíveis

6. Interface Web Profissional

A interface web é servida pelo FastAPI na porta 8080 usando o arquivo web_ui.html como template estático. O design prioriza clareza clínica: bolhas de chat diferenciadas, fontes visíveis por resposta e controle direto sobre o backend de inferência.

Funcionalidade	Descrição
Chat com bolhas diferenciadas	Consulta (azul), emergência (vermelho+alerta), sistema (cinza)
Painel de fontes	Documentos MedQuAD consultados exibidos ao lado de cada resposta
Badge de status do backend	Header mostra: OpenAI API Ollama gemma3:4b phi3-medassist — atualizado em tempo real
Troca de backend em tempo real	Modal 'Configurar Backend' — sem reiniciar o servidor
Disclaimer médico automático	Exibido em todas as respostas geradas pelo LLM
Bolha de emergência	Vermelho + ícone de alerta + SAMU 192 em destaque
Endpoint /api/config (FastAPI)	Recebe nova config do modal e atualiza os env vars em runtime
Endpoint /api/info (FastAPI)	Retorna backend ativo, modelo e status da API key
Endpoint /api/chat (FastAPI)	Processa mensagem → executa grafo LangGraph → retorna JSON

7. API REST (FastAPI)

O servidor FastAPI (web_app.py) expõe os endpoints abaixo na porta 8080. A documentação interativa Swagger está disponível em <http://localhost:8080/docs>.

Método	Endpoint	Descrição
GET	/	Interface web HTML (web_ui.html)
GET	/api/info	Status do backend ativo, modelo e API key configurada
POST	/api/chat	Processa mensagem → LangGraph → retorna resposta + fontes + flags
POST	/api/config	Atualiza backend em tempo real (sem reiniciar)
GET	/api/health	Healthcheck — retorna status do sistema

8. Avaliação e Resultados

8.1 Validação dos Componentes

Componente	Teste	Resultado
Adaptadores LoRA (fine-tuning)	validate_adapters.py	28/28 verificações APROVADAS
Qualidade dos dados	validate_data.py	0 duplicatas, 0 inválidas — APROVADA
Safety Guard	safety_guard.py — 5 cenários	5/5 cenários corretos
Fluxo LangGraph	graph.py — 4 casos de teste	Emergência, consulta, fora do escopo, prescrição
Pipeline RAG	chains.py + similarity_search	Top-3 docs com similaridade > 0.7 para queries médicas
Fine-tuning qualitativo	5 perguntas no Colab	5/5 respostas clinicamente adequadas

8.2 Comparativo com Outras Abordagens

Abordagem	Fine-tuning	Qualidade médica	Custo operacional
MedAssist (este projeto)	✓ QLoRA no Colab	Alta — MedQuAD + RAG	Ollama: zero / OpenAI: por token
LLM genérico (sem FT)	X Não	Média — sem domínio médico	Por token
RAG sem fine-tuning	X Não	Boa — depende do corpus	Por token ou Ollama
Fine-tuning sem RAG	✓	Alta no treino — lacunas no geral	Inferência com GPU

8.3 Métricas de Avaliação Automatizada

O módulo fase5_avaliacao/benchmark.py calcula métricas ROUGE, BLEU e F1 comparando as respostas do sistema com respostas de referência do dataset de validação (val.jsonl). O relatório técnico completo é gerado por generate_report.py em outputs/relatorio_tecnico_final.md.

Métrica	Descrição
ROUGE-1	Sobreposição de unigrams entre resposta gerada e referência
ROUGE-L	Longest common subsequence — captura estrutura sequencial
BLEU	Precisão de n-grams — mais exigente que ROUGE
F1 de tokens	Média harmônica de precisão e recall de tokens

9. Instalação e Uso

9.1 Pré-requisitos

- Python 3.10+ (recomendado 3.11)
- pip atualizado
- Para MODO 1: Chave de API da OpenAI (<https://platform.openai.com/api-keys>)
- Para MODO 2: Ollama instalado + gemma3:4b baixado (~3.3 GB)
- Para MODO 3: Ollama instalado + phi-3-mini-4k-instruct.Q4_K_M.gguf (~2.2 GB) baixado do Google Drive + ollama create phi3-medassist -f Modelfile
- Arquivos grandes (Drive): adapter_model.safetensors (~114 MB) e phi-3-mini-4k-instruct.Q4_K_M.gguf (~2.2 GB) — link no README Passo 5

9.2 Fluxo Sequencial de Execução

Passo	Comando
1. Clonar repositório	git clone <repo> && cd medassist
2. Criar ambiente virtual (Windows)	python -m venv venv && venv\Scripts\activate
2. Criar ambiente virtual (Mac/Linux)	python3 -m venv venv && source venv/bin/activate
3. Instalar dependências	pip install --upgrade pip && pip install -r requirements.txt
4. Configurar ambiente	cp .env.example .env # editar com MODO 1, 2 ou 3
5. Baixar arquivos do Google Drive	adapter_model.safetensors → outputs/model/ (todos os modos) phi-3-mini-4k-instruct.Q4_K_M.gguf → outputs/gguf/ (MODO 3)
6. [MODO 2] Instalar Ollama	curl -fsSL https://ollama.com/install.sh sh (Linux)
6. [MODO 2] Baixar modelo	ollama pull gemma3:4b
6. [MODO 2] Iniciar servidor	ollama serve (Linux — ou automático no Win/Mac)
7. [MODO 3] Registrar modelo fine-tunado	cd outputs/gguf && ollama create phi3-medassist -f Modelfile
7. [MODO 3] Verificar	ollama list # phi3-medassist deve aparecer
8. Explorar e preprocessar dados	python fase1_dados/explore_dataset.py
9. Preprocessar e dividir dataset	python fase1_dados/preprocess.py
10. Validar adaptadores LoRA	python fase2_finetuning/validate_adapters.py → 28/28
11. Criar vector store FAISS	python fase3_langchain/chains.py
12. Testar fluxo LangGraph	python fase3_langchain/graph.py
13. Iniciar interface web	python web_app.py → http://localhost:8080
14. [Opcional] Testar segurança	python fase4_seguranca/safety_guard.py
15. [Opcional] Gerar avaliação	python fase5_avaliacao/benchmark.py && python fase5_avaliacao/generate_report.py

10. Estrutura do Repositório

Arquivo / Pasta	Descrição
web_app.py	Servidor FastAPI — porta 8080 — serve interface e endpoints /api/*
web_ui.html	Interface web: chat + painel de fontes + modal de configuração de backend
config.py	Configurações globais: paths, LLM params, keywords de segurança
utils.py	Utilitários: encoding, logs, helpers
local_server.py	Servidor OpenAI-compatible para Phi-3-mini + LoRA (requer GPU)
requirements.txt	Dependências Python: langchain, langgraph, faiss, openai, fastapi, uvicorn...
.env.example	Template: MODO 1 (OpenAI), MODO 2 (Ollama gemma3:4b) e MODO 3 (phi3-medassist) comentados
data/medquad_ptbr.jsonl	5.274 pares Q&A médicos em PT-BR (corpus principal)
data/train.jsonl	4.747 amostras de treino (90%)
data/val.jsonl	527 amostras de validação (10%)
medquad_dataset/	Pipeline de dados: scripts de limpeza, tradução e documentação
medquad_dataset/clean_medquad.py	Limpeza e filtragem dos XMLs do MedQuAD → JSONL EN
medquad_dataset/translate_ptbr.py	Tradução EN → PT-BR via deep_translator
medquad_dataset/README.md	Documentação completa do pipeline de dados
outputs/model/	Adaptadores LoRA do fine-tuning — adapter_model.safetensors (~114 MB) via Google Drive
outputs/gguf/	MODO 3: phi-3-mini-4k-instruct.Q4_K_M.gguf (~2.2 GB) via Google Drive + Modelfile
outputs/vectorstore/	Vector store FAISS (index.faiss + index.pkl)
outputs/relatorio_tecnico_final.md	Relatório técnico gerado automaticamente
fase1_dados/explore_dataset.py	Análise exploratória: distribuição, comprimentos, categorias
fase1_dados/preprocess.py	Preprocessamento e split 90/10
fase1_dados/validate_data.py	Validação de qualidade: duplicatas, campos obrigatórios
fase2_finetuning/MedAssist_FineTuning_Colab.ipynb	Notebook Colab: QLoRA + Unsloth + Phi-3-mini
fase2_finetuning/validate_adapters.py	28 verificações automatizadas dos adaptadores LoRA
fase3_langchain/llm_backend.py	Backend unificado: OpenAI API, Ollama gemma3:4b e Ollama phi3-medassist (mesma interface)
fase3_langchain/chains.py	Pipeline RAG: embeddings + FAISS + recuperação
fase3_langchain/graph.py	Fluxo LangGraph: 6 nós + roteamento condicional

Arquivo / Pasta	Descrição
fase3_langchain/assistant.py	Interface de alto nível do assistente
fase4_seguranca/safety_guard.py	Guardrails: emergência, prescrição, fora do escopo
fase4_seguranca/logging_audit.py	Logging JSON em logs/audit.log
fase4_seguranca/explainability.py	Fontes MedQuAD + score de confiança
fase5_avaliacao/benchmark.py	Métricas ROUGE, BLEU, F1
fase5_avaliacao/generate_report.py	Geração do relatório técnico final
logs/audit.log	Log de interações em formato JSON Lines
README.md	Instruções completas: instalação, 3 modos, Google Drive, fluxo sequencial, troubleshooting
CHECKLIST.md	Checklist alinhado aos requisitos do TC Fase 3
OLLAMA_SETUP.md	Guia detalhado para configuração do MODO 2 (gemma3:4b) e MODO 3 (phi3-medassist via GGUF)

11. Integração: Fine-tuning + RAG + LangChain + LangGraph

Os quatro pilares tecnológicos do MedAssist são complementares e se integram em um pipeline coeso. O diagrama abaixo representa o fluxo de dados e as dependências entre os componentes:

Componente	Papel no Pipeline	Conexão com os demais
Fine-tuning (QLoRA)	Especializa o Phi-3-mini em linguagem médica PT-BR. Os adaptadores LoRA codificam o domínio médico no modelo base.	Adaptadores em outputs/model/ disponíveis via Google Drive. Via Células 12b/12c do Colab, são exportados como GGUF e registrados no Ollama como phi3-medassist (MODO 3).
RAG (FAISS + MedQuAD)	Recupera os 3 documentos mais relevantes do corpus médico para cada pergunta, fornecendo contexto factual ao LLM.	O contexto recuperado é injetado no prompt do LangChain antes de chamar o backend de inferência (MODO 1, 2 ou 3).
LangChain (chains.py)	Constrói o pipeline de recuperação: cria embeddings, indexa o corpus, realiza similarity search e formata os prompts com contexto RAG.	As chains são invocadas pelo LangGraph dentro do nó node_busca_rag, retornando docs e sources para o estado compartilhado.
LangGraph (graph.py)	Orquestra o fluxo completo: triagem → emergência ou RAG → geração → validação, com estado compartilhado entre nós e roteamento condicional.	Integra todos os outros: invoca RAG via chains.py, chama backend via llm_backend.py, aplica guardrails de segurança.

12. Decisões Arquiteturais

Decisão	Justificativa	Alternativas consideradas
Phi-3-mini como modelo de FT	3.8B parâmetros: cabe no Colab T4 com QLoRA; excelente suporte a chat template e PT	LLaMA-3 (maior, não cabe no T4 gratuito), Falcon (menor suporte a PT)
QLoRA via Unsloth	Reduz VRAM de ~16 GB para ~6 GB; throughput 30-50% maior que HF padrão	LoRA padrão (mais VRAM), full fine-tuning (inviável no T4)
MedQuAD como dataset	Público (NIH), sugerido pelo TC, cobertura ampla de condições médicas	PubMedQA (também sugerido, mais técnico/científico)
Tradução automática PT-BR	Adapta o dataset ao contexto brasileiro sem coleta manual de dados	Dados sintéticos gerados por LLM (menor fidelidade médica)
FAISS para vector store	Eficiente, sem servidor externo, funciona localmente	Chroma, Pinecone (requerem servidor ou cloud)
Três backends (OpenAI + Ollama + GGUF)	MODO 1: OpenAI para simplicidade; MODO 2: Ollama genérico offline e gratuito; MODO 3: modelo fine-tunado rodando via Ollama sem GPU	Somente local com GPU (restringiria o público)
GGUF via Ollama para MODO 3	Converte LoRA+base em GGUF quantizado (q4_k_m ~2.2 GB): roda o próprio modelo treinado localmente sem GPU, sem API key, offline	Inferência direta em RAM com GPU (requer ~8 GB VRAM, inviável sem hardware)
Arquivos grandes via Google Drive	adapter_model.safetensors (~114 MB) e phi-3-mini.Q4_K_M.gguf (~2.2 GB) excedem o limite do Git — distribuídos via Drive com link no README	Git LFS (pago acima de 1 GB), compressão ZIP (ainda grande demais)
FastAPI + HTML/CSS/JS inline	Interface leve sem dependência de framework frontend	React/Vue (overkill para demonstração)
LangGraph sobre LangChain puro	Roteamento condicional explícito — grafo de estados auditável	LangChain LCEL (menos visual para fluxos com branches)

13. Conclusão

O projeto MedAssist entrega um sistema completo de assistente virtual médico, atendendo a todos os requisitos obrigatórios do Tech Challenge Fase 3. Principais diferenciais:

- Fine-tuning real com QLoRA — Phi-3-mini especializado em português médico, adaptadores validados em 28/28 verificações
- Dataset MedQuAD PT-BR processado end-to-end — 16.000 XMLs brutos tratados em 5.274 pares de treino em português
- Pipeline RAG com LangChain — recuperação semântica em 5.274 documentos, top-3 por consulta, fontes visíveis
- Fluxo LangGraph auditável — 6 nós com roteamento condicional, emergência com bloqueio do LLM
- Segurança clínica em três camadas — Safety Guard (5/5), logging JSON, explainability com fontes
- Três backends intercambiáveis — OpenAI API, Ollama gemma3:4b e Phi-3-mini fine-tunado (phi3-medassist via GGUF), troca em tempo real sem reiniciar
- Interface web profissional — chat + painel de fontes + badge de status + modal de configuração
- Código modularizado em 5 fases — cada fase executável e testável independentemente

Possíveis evoluções: integração com PubMedQA para enriquecimento do corpus; LangSmith para observabilidade do grafo LangGraph; suporte a documentos clínicos do paciente via RAG multimodal; avaliação humana por especialistas médicos.

FIAP Pós-Tech — Inteligência Artificial para Devs | Tech Challenge Fase 3 | Maio / 2026